Laboratorio de Java / Practica 8
MPOO

Esquivel Cortes Iker Alonso 323004488
Hernandez Crisostomo Emilio 323165686

1. Sobrecarga contra sobrescritura

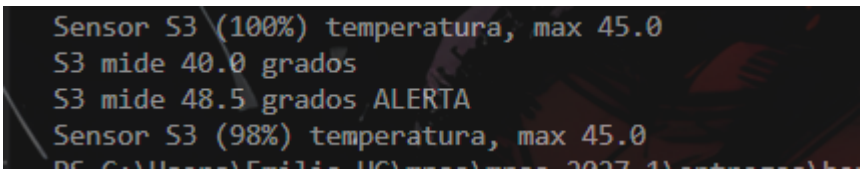
En mi código utilicé la sobrecarga en la clase Estacion al definir dos métodos llamados instalar: uno que recibe un objeto de tipo Sensor (instalar(Sensor s)) y otro que recibe un Actuador (instalar(Actuador a)). Tienen exactamente el mismo nombre, pero Java distingue cuál ejecutar según el tipo de argumento que se le envíe.
Por otro lado, la sobrescritura la apliqué en clases hijas como SensorTemperatura, donde volví a escribir el método medir() heredado de su clase padre Sensor. Mientras que en el padre el método es abstracto y no tiene cuerpo, en la clase hija se define la lógica exacta para calcular la señal sumándole la calibración.
La diferencia principal radica en que la sobrecarga se da dentro de la misma clase o jerarquía usando el mismo nombre pero cambiando los parámetros (la firma), mientras que la sobrescritura consiste en modificar el comportamiento de un método heredado manteniendo idénticos el nombre y los parámetros.

2. Composición contra agregación

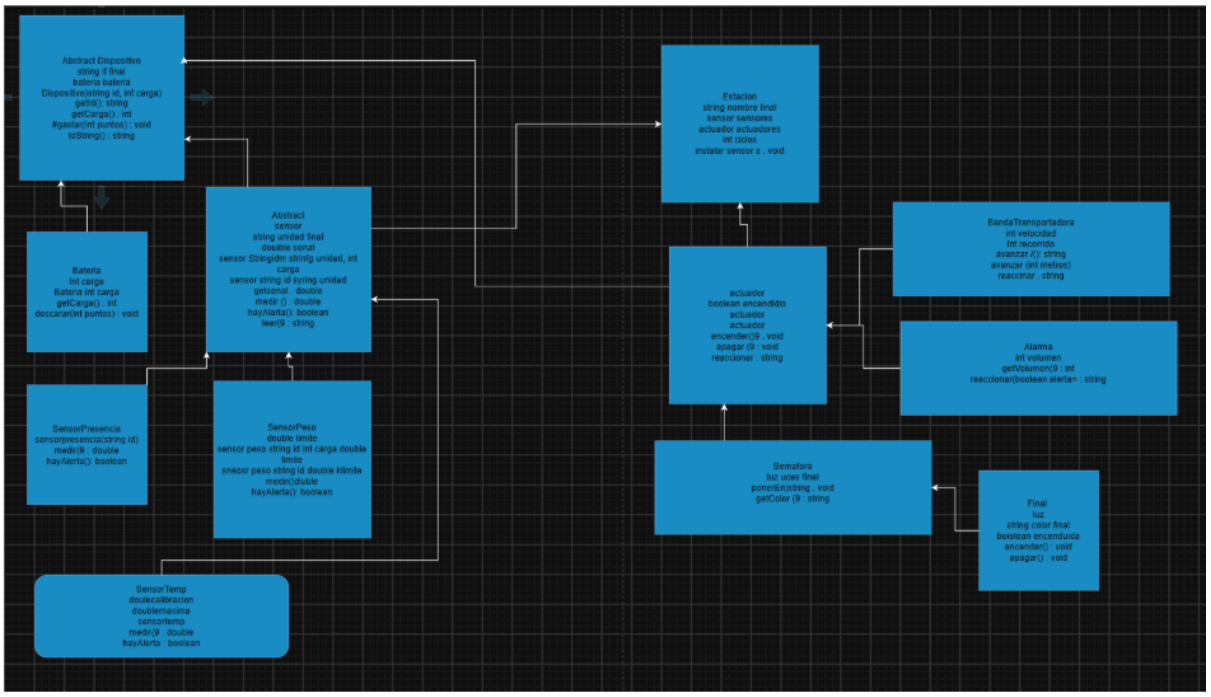
La relación entre Dispositivo y Bateria es de composición porque la batería es creada directamente dentro del dispositivo mediante un new Bateria(carga) en su constructor. Esto significa que la batería le pertenece de forma exclusiva y su ciclo de vida está atado al del dispositivo; si el dispositivo desaparece, la batería también.
En cambio, la relación entre Estacion y los sensores es de agregación. Los sensores se instancian externamente (en el Main) y la estación solo los almacena en un arreglo. Esto se comprobó claramente en el ciclo 3 cuando se retiró a S2 para mantenimiento: el sensor salió de la estación, pero no se destruyó, sino que conservó su estado y el desgaste de su batería (78%) en memoria para poder regresar y ser reinstalado después.

3. Modificadores

Al agregar la línea bascula.gastar(5); dentro de la clase Main, el compilador arroja un error que indica que el método no es visible o tiene acceso protegido:
gastar(int) has protected access in Dispositivo
¿Por qué SensorPeso sí puede usar gastar? Porque SensorPeso es una clase hija que hereda de Dispositivo (a través de Sensor), y el método gastar fue declarado con el modificador protected. Esto permite que cualquier subclase pueda invocarlo



directamente, aunque pertenezca a un paquete diferente (en este caso, sensores frente a dispositivos).
¿Por qué Main no puede usarlo? Porque la clase Main no hereda de Dispositivo. El modificador protected restringe el acceso únicamente a las clases hijas y a las que comparten el mismo paquete, por lo que las clases externas sin parentesco no tienen permitido llamar a este método.




```
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 grados
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 grados
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 grados ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
PS C:\Users\Emilio HC\mpoo\mpoo-2027-1\entregas\hernandez_emilio\p08>
```